

System Design Document (SDD)

CleanTrack — Arsitektur Sistem

Versi 2.0 — arsitektur PostgreSQL + NestJS, mencakup modul foto, berat, dan tracking kurir.

1. Gambaran Arsitektur

CleanTrack dibangun sebagai backend modular berbasis NestJS (Node.js/TypeScript) dengan PostgreSQL sebagai database utama diakses melalui Prisma ORM. Arsitektur mengikuti pola shared-database multi-tenancy: seluruh tenant berbagi satu database dan skema tabel yang sama, dengan isolasi data dijamin melalui kolom tenant_id pada hampir seluruh tabel dan middleware/guard yang memvalidasi kepemilikan data di setiap request.

Frontend terdiri dari dua aplikasi terpisah: web admin/dashboard (Next.js) untuk Admin/Owner/Operator/Kasir, dan aplikasi mobile (React Native/Expo) untuk Customer dan Kurir.

2. Stack Teknologi

Layer	Teknologi
Database	PostgreSQL
ORM	Prisma
Backend Framework	NestJS (Node.js, TypeScript)
Autentikasi	JWT (access + refresh token)
Cache & Queue	Redis + BullMQ
Storage Foto	Cloudflare R2 / AWS S3
Realtime	WebSocket (native NestJS Gateway) untuk lokasi kurir & update status order
Payment Gateway	Midtrans / Xendit
WhatsApp	WhatsApp Cloud API (resmi, via Meta atau BSP seperti Qontak/Wablas)
Push Notification	Firebase Cloud Messaging (FCM)
Maps	Google Maps Platform (Geocoding, Distance Matrix, Directions)
Frontend Web	Next.js + TypeScript + Tailwind + shadcn/ui
Frontend Mobile	React Native + Expo
Infra	Docker, Railway/Fly.io (awal) → AWS/GCP (scale), GitHub Actions CI/CD

3. Struktur Modul NestJS

Setiap domain fungsional dipisah menjadi module NestJS independen, saling berkomunikasi melalui service layer tanpa saling bergantung langsung pada Prisma model milik module lain:

```
src/
├─ tenant/           (Tenant, Branch)
├─ auth/             (JWT, guard, role-based access)
├─ employee/         (Employee, role management)
├─ customer/         (Customer, CustomerAddress)
├─ courier/          (Courier, CourierLocationLog, live tracking gateway)
├─ order/            (Order, OrderItem, OrderStatusLog, state machine)
├─ weight-verification/ (perbandingan berat, trigger mismatch)
├─ photo-verification/ (upload & validasi foto wajib per status)
├─ membership/       (Membership, Discount)
├─ service-catalog/  (Service, TenantCourierRules)
```

— payment/	(Payment, integrasi Midtrans/Xendit, webhook)
— notification/	(WhatsappModule, FCM push, BullMQ processor)
— maps/	(Google Maps: geocoding, distance matrix, directions)
— storage/	(upload foto ke R2/S3)

4. Strategi Multi-Tenancy

- Pola: shared database + kolom tenant_id (bukan schema-per-tenant), dipilih karena lebih murah dioperasikan dan lebih mudah dikelola migrasinya untuk skala SaaS awal-menengah.
- Setiap request terautentikasi membawa tenant_id (diekstrak dari JWT payload employee/customer), dan seluruh query Prisma pada service layer wajib menyertakan filter tenant_id.
- Guard tingkat request (TenantGuard) memverifikasi bahwa resource yang diakses (order, customer, dsb.) benar-benar milik tenant yang sedang login, mencegah akses lintas tenant (IDOR).
- Index komposit (tenant_id, branch_id) pada tabel-tabel master memastikan query tetap efisien meski data seluruh tenant berada dalam satu tabel yang sama.

5. Desain State Machine Order

Transisi status order divalidasi secara eksplisit oleh OrderStateMachineService, bukan hanya diperbarui langsung dari controller, untuk mencegah lompatan status yang tidak valid:

pending → pickup → received → process → delivery → done

(semua status di atas dapat berpindah ke 'cancelled' sebelum status 'process' dimulai)

- Transisi ke 'pickup' dan 'delivery' (final) memerlukan photo_url wajib — divalidasi oleh PhotoVerificationService sebelum state machine mengizinkan perpindahan.
- Transisi ke 'received' memerlukan actual_weight terisi dan foto bukti timbangan — divalidasi oleh WeightVerificationService.
- Jika weight_status = 'mismatched' setelah transisi ke 'received', state machine menahan order pada status tersebut hingga weight_status berubah menjadi 'confirmed' oleh alur konfirmasi pelanggan.

6. Alur Integrasi Pihak Ketiga

6.1 Pembayaran

Customer bayar → Webhook Midtrans/Xendit → PaymentController

- update Payment + Order.payment_status (Prisma)
- push job ke BullMQ Queue
- FCM Push (in-app) + WhatsApp Cloud API (kirim nota)

6.2 Verifikasi Berat Mismatch

Kasir input actual_weight → WeightVerificationService bandingkan dgn estimasi

- jika mismatch: recalculate subtotal/total, set weight_status='mismatched'
- push job ke BullMQ → kirim WhatsApp interaktif (tombol Ya/Tidak)
- webhook balasan WA / konfirmasi in-app → weight_status='confirmed'
- OrderStateMachineService lanjutkan status ke 'process'

6.3 Live Tracking Kurir

App Kurir → ping lokasi berkala → CourierController (REST) atau WS Gateway
→ simpan ke CourierLocationLog (courier_id, order_id, long, lat)
→ broadcast via WebSocket Gateway ke customer yang sedang tracking order tsb.

7. Desain Keamanan

- Autentikasi JWT dengan access token berumur pendek dan refresh token tersimpan aman (httpOnly cookie untuk web, secure storage untuk mobile).
- Password pegawai/customer di-hash menggunakan bcrypt/argon2, tidak ada fallback PIN hardcoded pada endpoint apapun.
- Seluruh endpoint yang mengubah data (order, pembayaran, status) memvalidasi kepemilikan tenant/cabang/order sebelum eksekusi (mencegah IDOR).
- Endpoint reset password wajib melalui verifikasi (OTP WhatsApp atau email token), tidak boleh diakses tanpa autentikasi/verifikasi kepemilikan akun.
- File foto (bukti pickup/received/delivery) disimpan di storage dengan URL bertanda waktu terbatas (signed URL) untuk mencegah akses publik tanpa otorisasi.
- Rate limiting pada endpoint publik (login, webhook) untuk mencegah abuse.

8. Pertimbangan Skalabilitas

- courier_location_log berpotensi tumbuh cepat karena ping berkala; direkomendasikan strategi retensi (purge data lebih dari N hari) atau partisi tabel per bulan.
- Notification (WhatsApp, FCM) sepenuhnya melalui job queue asinkron agar tidak membebani response time endpoint utama.
- Query pelaporan (mismatch dashboard, laporan layanan terlaris) memanfaatkan index khusus (idx_order_weight_status, idx_order_item_service) agar tetap performan meski volume order bertambah.